

ASKIFY

A Project Report

Submitted By

Sk. Shareef

2203031241205

Smd. Ubaid

2203031241204

B. Venkata Vamshi

2203031240225

E. Yogesh

2203031241506

in Partial Fulfilment For the Award of

the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Prof. Shaik Mohd Maaz Naimoddin



VADODARA

August – 2025



PARUL UNIVERSITY

CERTIFICATE

This is to Certify that Project - II (203105400) of 7th Semester entitled “**ASKIFY**” of Group No. **AI-ASMP-88** has been successfully completed by

- SK.SHAREEF - 2203031241205
- SMD. UBAID - 2203031241204
- B. VENKATA VAMSHI - 2203031240225
- E.YOGESH - 2203031241506

under my guidance in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science & Engineering of Parul University in Academic Year 2023- 2024.

Date of Submission : _____

Prof. Shaik Mohd Maaz Naimoddin

Project Guide

Project Coordinator:-

Prof. Shaik Mohd Maaz Naimoddin

Dr. Amit Barve,

Head of Department,

CSE, PIET,

Parul University.

ACKNOWLEDGEMENT

“Small daily improvements over time lead to stunning results.”

— Robin Sharma

We would like to take this opportunity to sincerely thank everyone who supported us in bringing the Askify project to completion.

First and foremost, we express our heartfelt gratitude to our project guide, **Prof. Shaik Mohd Maaz Naimoddin**, for his constant encouragement, guidance, and constructive feedback throughout the journey. His valuable insights not only shaped the technical direction of our project but also motivated us to push our limits.

We are also deeply thankful to our respected Head of Department, **Dr. Amit Barve**, and all the faculty members of the **CSE Department**, PIET, for their continuous support, advice, and encouragement, which helped us refine our work and improve the quality of the project.

A special note of appreciation goes to our teammates — Shareef, Ubaid, Vamshi, and Yogesh — whose dedication, patience, and collaborative spirit made this project a reality. Each member's contribution was equally important in overcoming the challenges we faced during development.

We would also like to thank the individuals who tested our application and provided genuine feedback. Their suggestions were invaluable in making Askify more practical and user-friendly.

Finally, we owe our deepest gratitude to our families and friends, who have been our constant pillars of strength. Their patience, encouragement, and belief in us gave us the confidence to stay focused and complete this project successfully.

With sincere appreciation, we acknowledge the collective effort of everyone who contributed to this journey. Without their support, completing this project would not have been possible.

Sk.Shareef -2203031241205

Smd.Ubaid-2203031241204

B.Venkata Vamshi-2203031240225

E.Yogesh-210304124426

CSE, PIET

Parul University,

Vadodara

ABSTRACT

The Askify project aims to simplify the process of extracting knowledge from large PDF documents by providing a backend service that enables users to ask questions directly from the content of uploaded files. This report highlights the motivation, objectives, design, and technical implementation of Askify, along with its scope for future improvements.

In today's academic and professional world, individuals often rely on PDFs for research papers, technical manuals, reports, and study material. However, going through lengthy documents to locate specific information can be time-consuming and overwhelming. To address this challenge, Askify was developed as a solution that allows users to upload a PDF and interact with it in a conversational manner. Instead of manually searching for keywords or reading through hundreds of pages, users can simply pose questions and receive precise answers.

Askify leverages LangChain for natural language understanding and integrates ChromaDB for efficient storage and retrieval of document embeddings. The system follows a Retrieval-Augmented Generation (RAG) approach, combining context-based retrieval with advanced language models to deliver accurate and context-aware responses. The backend is powered by FastAPI for seamless API interactions, while SQLite stores extracted text for lightweight and reliable database management. Furthermore, WebSockets enable real-time communication, making the question-answering experience smooth and interactive.

Beyond its technical capabilities, Askify has a wide range of practical applications. Students can use it to quickly revise or prepare for exams by asking questions from their study material. Researchers can extract key insights from lengthy academic papers without reading them word by word. Professionals working with contracts, legal documents, or technical manuals can use Askify to retrieve relevant information instantly.

Overall, Askify represents a significant step toward smarter document interaction by bridging the gap between static PDFs and dynamic knowledge access. By combining modern natural language processing with efficient retrieval techniques, the system not only saves time but also improves productivity and decision-making. With further enhancements, Askify has the potential to become a valuable AI-powered tool in the fields of education, research, and professional document management.

Key Points: PDF Question Answering, LangChain, RAG, ChromaDB, FastAPI, NLP

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Tables	ix
List of Figures	xi
1 INTRODUCTION	1
1.1 Background and Problem Definition	1
1.2 Purpose and Scope of the Project	1
1.3 Technical Framework	2
1.4 User-Centered Design Model	2
1.5 Value for Users and Practical Applications	3
2 LITERATURE SURVEY	4
2.1 PDF Information Retrieval Systems	4
2.2 Question Answering Models with NLP	5
2.3 Retrieval-Augmented Generation (RAG) in Document Processing	6
2.4 AI-Powered Q&A Assistants and Chatbots	7
2.5 Database-Backed Search and Document Embedding Techniques	8
2.6 Role of LangChain in NLP Applications	9

2.7	ChromaDB for Vector Similarity Search.....	10
2.8	FastAPI and WebSocket for Real-Time Communication.....	11
2.9	Ethical Considerations in AI-based Information Retrieval.....	12
3	ANALYSIS / SOFTWARE REQUIREMENTS SPECIFICATION (SRS)	24
3.1	Introduction.....	24
3.1.1	Purpose.....	24
3.1.2	Terms	24
3.1.3	Intended Audience and Readers.....	24
3.1.4	Product Scope.....	24
3.1.5	References	25
3.2	General	25
3.2.1	Products.....	25
3.2.2	Product Features	25
3.2.3	User groups and characteristics.....	26
3.2.4	Operating environment.....	26
3.2.5	Design and Functionality.....	26
3.2.6	User Information.....	27
3.2.7	Assumptions and Dependencies.....	27
3.3	External Interface Requirements.....	27
3.3.1	User Interface.....	27
3.3.2	Hardware Interface.....	28
3.3.3	Software Interface	28
3.3.4	Communication.....	28
3.4	Functions and requirements	28
3.4.1	Functional requirements	28

4	SYSTEM DESIGN	31
4.1	System Architecture	31
4.2	User Interface Design	33
4.3	Database Design.....	35
5	METHODOLOGY	37
5.1	Technology Stack	37
5.2	Development Process	37
5.3	Testing and Evaluation.....	39
6	IMPLEMENTATION	40
6.1	Introduction.....	40
6.2	Technology Stack	40
6.3	Back End Development (FastAPI, WebSocket).....	40
6.4	Document Processing and PDF Parsing	43
6.5	Integration of LangChain and ChromaDB	43
6.6	Challenges and Solutions.....	43
7	TESTING	44
7.1	Introduction.....	44
7.2	Unit Testing	44
7.3	Integration Testing	44
7.4	User Acceptance Testing	46
7.5	Test Cases and Results.....	46
7.6	Performance Testing.....	47
7.7	Bug Tracking and Resolution.....	47
7.8	Final Testing and Validation	47

8	CONCLUSION	49
8.1	Summary of Results	49
8.2	Key Findings	49
8.3	Challenges	49
8.4	Lessons Learned.....	49
8.5	Future Directions.....	50
8.6	Conclusion	50
9	FUTURE WORK	51

List of Tables

2.1	Existing PDF Question-Answering Systems	4
2.2	Comparative Study of NLP-based Document Processing Approaches	5
2.3	Applications of LangChain in NLP and Q&A Systems	6
2.4	Studies on Vector Databases for Document Retrieval (ChromaDB, FAISS, etc.).....	7
2.5	Role of Real-Time Communication (WebSocket) in Interactive Applications	8
2.6	Ethical and Legal Considerations in AI-based Information Retrieval.....	9
2.7	Functional Requirements of Askify	10
2.8	Non-Functional Requirements of Askify	11
2.9	User Groups and Characteristics.....	12
2.10	System Architecture Components	13
2.11	Database Schema (SQLite for PDF Data Storage).....	14
2.12	Use Case Scenarios	15
2.13	Unit Testing of Core Modules.....	16
2.14	Integration Testing Results.....	17
2.15	User Acceptance Testing Outcomes	18
2.16	Performance and Load Testing Results.....	19
2.17	Bug Tracking and Resolution Summary.....	20
4.1	Database Schema.....	35

List of Figures

4.1	UML Diagram.....	32
4.2	Sequence Diagram	33
4.3	DFD Level-0	34
4.4	DFD Level-1	34
4.5	ER Model.....	36
4.6	Activity Diagram.....	36
6.1	API Upload Endpoint Response (Postman Screenshot)	41
6.2	WebSocket Communication Flow	41
6.3	Sample Console Output of Q&A.....	42
6.4	Database Entries in SQLite	42
7.1	Unit Testing of PDF Parsing Functions.....	45
7.2	Unit Testing of Q&A API Endpoints.....	45
7.3	Integration Testing: LangChain + ChromaDB	47
7.4	Performance Testing Results (Response Time Graph).....	48

Chapter 1

INTRODUCTION

1.1 Background and Problem Definition

The growing reliance on digital documents has transformed the way information is created, stored, and shared. From research articles and technical manuals to legal contracts and educational material, a significant portion of modern knowledge exists in the form of PDF files. While this format is widely accepted for preserving structure and readability, it also presents a challenge: extracting specific information from lengthy documents can be time-consuming and frustrating.

Students often struggle to quickly revise from study materials, researchers find it difficult to locate precise information in large academic papers, and professionals waste valuable time scanning through contracts or reports. Traditional search functions within PDFs are limited, as they rely on exact keywords rather than contextual understanding. This gap between information availability and information accessibility is what inspired the creation of Askify.

Askify addresses this challenge by providing an intelligent question-answering service for PDFs. Instead of forcing users to manually scan or search for text, the system allows them to upload a document and simply ask questions in natural language. By combining advanced natural language processing (NLP) techniques with efficient document retrieval, Askify transforms static PDFs into interactive knowledge sources.

1.2 Purpose and Scope of the Project

The purpose of Askify is to bridge the gap between static document storage and intelligent information retrieval. Its core objective is to design a backend service where users can upload PDF documents and interact with them as if they were conversing with a knowledgeable assistant.

The scope of the project goes beyond basic text search. Askify is capable of:

1. Extracting text from PDF files.
2. Generating document embeddings for semantic similarity search.
3. Answering user queries in real time using Retrieval-Augmented Generation (RAG).
4. Providing accurate, context-aware responses rather than keyword matches.

This makes Askify useful across multiple domains. In academics, students can upload textbooks or notes and quickly revise by asking questions. In research, it allows scholars to extract precise insights from lengthy publications. In the corporate world, it helps professionals save time by retrieving key clauses or summaries from contracts and policies.

The system is designed to be lightweight yet scalable, ensuring that it can run efficiently on smaller setups while still being extendable to enterprise-scale applications in the future.

1.3 Technical Framework

Askify is built on a robust technical foundation that integrates modern NLP frameworks with efficient data storage and real-time communication protocols. The architecture includes the following key components:

- **FastAPI:** Provides the backbone for the REST and WebSocket APIs, ensuring scalability and performance.
- **PyPDF2:** Used for parsing and extracting raw text from uploaded PDFs.
- **SQLite:** A lightweight database for storing processed document content.
- **LangChain:** Powers the natural language processing, query interpretation, and response generation.
- **ChromaDB:** Handles vector embeddings and similarity searches, enabling the system to retrieve contextually relevant information.
- **WebSocket Communication:** Ensures interactive, real-time Q&A between the user and the backend system.

By integrating these technologies, Askify is able to provide a smooth and responsive user experience. The use of RAG (Retrieval-Augmented Generation) further enhances accuracy by combining knowledge retrieval with intelligent language generation.

1.4 User-Centered Design Model

Askify is developed with a **user-first approach**, ensuring that the system is not only technically sound but also practical and easy to use. The design focuses on minimizing complexity while maximizing accessibility. Users only need to perform two simple actions: **upload a PDF** and **ask questions**.

The system hides all the backend complexity of text extraction, database management, and vector search. This ensures that both technical and non-technical users can benefit from the service without requiring advanced knowledge of AI or programming.

Furthermore, Askify is designed to adapt based on user feedback. During testing, users were encouraged to try real-world use cases such as exam preparation or policy document review, and their feedback was used to refine the response accuracy and interaction flow. This continuous improvement model ensures that Askify evolves with user needs, making it increasingly relevant and reliable over time.

1.5 Value for Users and Practical Applications

The true value of Askify lies in the time and effort it saves. By allowing direct interaction with documents, users no longer need to spend hours scrolling through pages of text. Instead, they can instantly receive accurate answers to their questions.

For students, this means faster revision and more effective exam preparation. For researchers, it provides a way to quickly extract key arguments, results, or references from long academic papers. For professionals, it enables faster decision-making by retrieving specific details from technical reports, contracts, or business documents.

Beyond individual use, Askify also holds potential for integration into larger platforms, such as educational learning systems, corporate knowledge bases, and digital libraries. With further enhancements, features such as multi-document analysis, voice-based Q&A, and support for additional file formats can be introduced, expanding the system's reach and applicability.

In summary, Askify represents a significant step toward making information retrieval smarter and more user-friendly. By combining NLP, retrieval systems, and real-time interaction, it turns static PDFs into dynamic, accessible knowledge hubs, providing meaningful value to students, researchers, and professionals alike.

Chapter 2

LITERATURE SURVEY

2.1 PDF Information Retrieval Systems

Student Name:	Shaik Shareef		
Enrollment No:	2203031241205	Branch:	CSE
Title Of Jounral Paper:	A Comprehensive Survey of Retrieval-Augmented Generation		
Authors:	Gupta, S., Ranjan, R., Singh, S. N.		
Journal/Conference:	arXiv preprint, arXiv:2410.12837. (Cornell University)		
Volume/Issue:	2024	Pages:	18

Table 2.1: PDF Information Retrieval systems

Abstract: This paper presents a comprehensive study of Retrieval-Augmented Generation (RAG), tracing its evolution from foundational concepts to the current state of the art. RAG combines retrieval mechanisms with generative language models to enhance the accuracy of outputs, addressing key limitations of LLMs. The study explores the basic architecture of RAG, focusing on how retrieval and generation are integrated to handle knowledge-intensive tasks. A detailed review of the significant technological advancements in RAG is provided, including key innovations in retrieval-augmented language models and applications across various domains such as question-answering, summarization, and knowledge-based tasks. Recent research breakthroughs are discussed, highlighting novel methods for improving retrieval efficiency. Furthermore, the paper examines ongoing challenges such as scalability, bias, and ethical concerns in deployment. Future research directions are proposed, focusing on improving the robustness of RAG models, expanding the scope of application of RAG models, and addressing societal implications. This survey aims to serve as a foundational resource for researchers and practitioners in understanding the potential of RAG and its trajectory in natural language processing. [3].

2.2 Question Answering Models with NLP

Student Name:	Shaik Shareef		
Enrollment No:	2203031241205	Branch:	CSE
Title Of Journal Paper:	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks		
Authors:	Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W-t., Rocktäschel, T., Riedel, S., Kiela, D.		
Journal/Conference:	arXiv preprint, arXiv:2005.11401 (Cornell University)		
Volume/Issue:	2020	Pages:	19

Table 2.2: Question Answering Models with NLP

Abstract: Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory can overcome this issue, but have so far been only investigated for extractive downstream tasks. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) -- models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We compare two RAG formulations, one which conditions on the same retrieved passages across the whole generated sequence, the other can use different passages per token. We fine-tune and evaluate our models on a wide range of knowledge-intensive NLP tasks and set the state-of-the-art on three open domain QA tasks, outperforming parametric seq2seq models and task-specific retrieve-and-extract architectures. For language generation tasks, we find that RAG models generate more specific, diverse and factual language than a state-of-the-art parametric-only seq2seq baseline. [7].

2.3 Retrieval-Augmented Generation (RAG) in Document Processing

Student Name:	Shaik Shareef		
Enrollment No:	2203031241205	Branch:	CSE
Title Of Journal Paper:	A Survey on Knowledge-Oriented Retrieval-Augmented Generation		
Authors:	Cheng, M., Luo, Y., Ouyang, J., Liu, Q., Liu, H., Li, L., Yu, S., Zhang, B., Cao, J., Ma, J., Wang, D.		
Journal/Conference:	arXiv preprint, arXiv:2503.10677.(Cornell University)		
Volume/Issue:	2025	Pages:	50

Table 2.3: E-Commerce Price Comparison Website Using Web Scraping

Abstract: Retrieval-Augmented Generation (RAG) has gained significant attention in recent years for its potential to enhance natural language understanding and generation by combining large-scale retrieval systems with generative models. RAG leverages external knowledge sources, such as documents, databases, or structured data, to improve model performance and generate more accurate and contextually relevant outputs. This survey aims to provide a comprehensive overview of RAG by examining its fundamental components, including retrieval mechanisms, generation processes, and the integration between the two. We discuss the key characteristics of RAG, such as its ability to augment generative models with dynamic external knowledge, and the challenges associated with aligning retrieved information with generative objectives. We also present a taxonomy that categorizes RAG methods, ranging from basic retrieval-augmented approaches to more advanced models incorporating multi-modal data and reasoning capabilities. Additionally, we review the evaluation benchmarks and datasets commonly used to assess RAG systems, along with a detailed exploration of its applications in fields such as question answering, summarization, and information retrieval. Finally, we highlight emerging research directions and opportunities for improving RAG systems, such as enhanced retrieval efficiency, model interpretability, and domain-specific adaptations. This paper concludes by outlining the prospects for RAG in addressing real-world challenges and its potential to drive further advancements in natural language processing. [1].

2.4 AI-Powered Q&A Assistants and Chatbots

Student Name:	Shaik Mohammed Ubaid		
Enrollment No:	2203031241204	Branch:	CSE
Title Of Journal Paper:	An Analysis of Large Language Models and LangChain in Mathematics Education		
Authors:	Soygazi, F., & Oguz, D.		
Journal/Conference:	The 7th International Conference on Advances in Artificial Intelligence (ICAAI 2023) At: Istanbul		
Volume/Issue:	2023	Pages:	4

Table 2.4: AI-Powered Q&A Assistants and Chatbots

Abstract: The development of large language models (LLMs) has led to the consideration of new approaches, particularly in education. Word problems, especially in subjects like mathematics, and the need to solve these problems by collectively addressing specific stages of reasoning, have raised the question of whether LLMs can be successful in this area as well. In our study, we conducted analyses by asking mathematics questions especially related to word problems using ChatGPT, which is based on the latest language models like Generative Pretrained Transformer (GPT). Additionally, we compared the correct and incorrect answers by posing the same questions to LLMMathChain, a mathematics-specific LLM based on the latest language models like LangChain. It was observed that the answers obtained were more successful with ChatGPT (GPT 3.5), particularly in the field of mathematics. However, both language models were found to be below expectations, particularly in word problems, and suggestions for improvement were provided. CCS CONCEPTS • Computing methodologies → Artificial intelligence. [8].

2.5 Database-Backed Search and Document Embedding Techniques

Student Name:	Shaik Mohammed Ubaid		
Enrollment No:	2203031241204	Branch:	CSE
Title Of Journal Paper:	Information Retrieval: Recent Advances and Beyond		
Authors:	Hambarde, K. A.		
Journal/Conference:	arXiv preprint, arXiv:2301.08801.(Cornell University)		
Volume/Issue:	2023	Pages:	26

Table 2.5: Database-Backend Search and Document Embedding Techniques

Abstract: In this paper, we provide a detailed overview of the models used for information retrieval in the first and second stages of the typical processing chain. We discuss the current state-of-the-art models, including methods based on terms, semantic retrieval, and neural. Additionally, we delve into the key topics related to the learning process of these models. This way, this survey offers a comprehensive understanding of the field and is of interest for researchers and practitioners entering/working in the information retrieval domain. [13].

2.6 Role of LangChain in NLP Applications

Student Name:	Bujumekala Venkata Vamshi		
Enrollment No:	2203031240225	Branch:	CSE
Title Of Journal Paper:	Chatbot Development Using LangChain: A Case Study to Foster Critical Thinking and Creativity		
Authors:	Farinetti, L.		
Journal/Conference:	ITiCSE 2024: Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1		
Volume/Issue:	2024	Pages:	7

Table 2.6: Role of LangChain in NLP Applications

Abstract: Critical thinking and creativity are fundamental skills for engineers and computer scientists. The emergence of Large Language Models (LLMs) able to create chatbots that use natural language is an opportunity for educators to foster these skills. The well-known risk of generative AI for potential misinformation offers fertile ground to practice critical thinking. This paper describes a hands-on experience within a database course, where students had to develop a chatbot using the LangChain framework, and to evaluate it from different points of view. The students were free to choose the domain of their chatbot. The learning goal was twofold: on the one hand, to make them practice with state-of-the-art technologies, and on the other hand to stimulate critical analysis on their output. The paper discusses the students' evaluation of the chatbots under several metrics, including document retrieval, syntax and grammar accuracy, semantic relevance and information reliability. Students' assessments were also compared to the teachers' ones, to gain an insight on the critical attitude of the students and to offer a ground for discussion. The experience was stimulating and appreciated by the students. The final results highlight that the majority of students successfully produced chatbot responses that were grammatically and syntactically correct, and that consistently extracted pertinent sections from documents, yielding semantically relevant outputs. Despite these achievements, a significant portion of students expressed reservations about the reliability of the chatbot's responses to prompts, gaining awareness of LLMs' capability to generate responses that make sense to humans but may be potentially misleading. [10].

2.7 ChromaDB for Vector Similarity Search

Student Name:	Bujumekala Venkata Vamshi		
Enrollment No:	2203031240225	Branch:	CSE
Title Of Journal Paper:	Embeddings and Vector Databases With ChromaDB		
Authors:	Harrison Hoffman		
Journal/Conference:	Open Scourse		
Volume/Issue:	2025	Pages:	20

Table 2.7: ChromaDB for Vector Similarity Search

Abstract: The era of [large language models](#) (LLMs) is here, bringing with it rapidly evolving libraries like [ChromaDB](#) that help augment LLM applications. You’ve most likely heard of chatbots like OpenAI’s [ChatGPT](#), and perhaps you’ve even experienced their remarkable ability to reason about [natural language processing \(NLP\)](#) problems.

Modern LLMs, while imperfect, can accurately solve a wide range of problems and provide correct answers to many questions. But, due to the limits of their training and the number of text tokens they can process, LLMs aren’t a silver bullet for all tasks.

You wouldn’t expect an LLM to provide relevant responses about topics that don’t appear in their training data. For example, if you asked ChatGPT to summarize information in confidential company documents, then you’d be out of luck. You could show some of these documents to ChatGPT, but there’s a limited number of documents that you can upload before you exceed ChatGPT’s maximum number of tokens. How would you select documents to show ChatGPT?

To address these shortcomings and scale your LLM applications, one great option is to use a vector database like ChromaDB. A **vector database** allows you to store encoded unstructured objects, like text, as lists of numbers that you can compare to one another. You can, for example, find a collection of documents relevant to a question that you want an LLM to answer. [2].

2.8 FastAPI and WebSocket for Real-Time Communication

Student Name:	Eranna Gari Yogesh		
Enrollment No:	2203031241506	Branch:	CSE
Title Of Journal Paper:	A Primer on Developing LLM Apps Fast, focusing on LangChain and real-time architecture.		
Authors:	As-Proceeding.com		
Journal/Conference:	International Conference on Applied Engineering and Natural Sciences		
Volume/Issue:	2023	Pages:	6

Table 2.8: FastAPI and WebSocket for Real-Time Communication

Abstract: This study focuses on the utilization of Large Language Models (LLMs) for the rapid development of applications, with a spotlight on LangChain, an open-source software library. LLMs have been rapidly adopted due to their capabilities in a range of tasks, including essay composition, code writing, explanation, and debugging, with OpenAI's ChatGPT popularizing their usage among millions of users. The crux of the study centers around LangChain, designed to expedite the development of bespoke AI applications using LLMs. LangChain has been widely recognized in the AI community for its ability to seamlessly interact with various data sources and applications. The paper provides an examination of LangChain's core features, including its components and chains, acting as modular abstractions and customizable, use-case-specific pipelines, respectively. Through a series of practical examples, the study elucidates the potential of this framework in fostering the swift development of LLM-based applications. [9].

2.9 Ethical Considerations in AI-based Information Retrieval

Student Name:	Eranna Gari Yogesh		
Enrollment No:	2203031241506	Branch:	CSE
Title Of Journal Paper:	A Systematic Review of Fairness, Accountability, Transparency, and Ethics in Information Retrieval		
Authors:	Nolwenn Bernard, Krisztian Balog		
Journal/Conference:	ACM Computing Surveys, Volume 57, Issue 6 Article No.: 136		
Volume/Issue:	2025	Pages:	29

Table 2.9: Ethical Considerations in AI-based Information Retrieval

Abstract: We live in an information society that strongly relies on information retrieval systems, such as search engines and conversational assistants. Consequently, the trustworthiness of these systems is of critical importance and has attracted significant research attention in recent years. In this work, we perform a systematic literature review of the field of fairness, accountability, transparency, and ethics in information retrieval. In particular, we investigate the definitions, approaches, and evaluation methodologies proposed to build trustworthy information retrieval systems. This review reveals the lack of standard definitions, arguably due to the multi-dimensional nature of the different notions. In terms of approaches, most of the work focuses on building either a fair or a transparent information retrieval system. As for evaluation, fairness is often assessed by means of automatic evaluation, while accountability and transparency are most commonly evaluated using audits and user studies. Based on the surveyed literature, we develop taxonomies of requirements for the different notions, and further use these taxonomies to propose practical definitions to quantify the degree to which an information retrieval system satisfies a given notion. Finally, we discuss challenges that have yet to be solved for information retrieval systems to be trustworthy. [12].

Chapter 3

ANALYSIS / SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

3.1 Introduction

3.1.1 Purpose

This document outlines the functional and non-functional requirements for **Askify**, a PDF-based Question-Answering platform. Askify enables users to upload academic or reference PDFs and query them in natural language. The system integrates **LangChain** for NLP, **ChromaDB** for semantic search, and **FastAPI with WebSockets** for real-time communication. The purpose of this SRS is to provide a clear technical and functional blueprint for developers, project coordinators, and evaluators to ensure the system is implemented consistently and meets its intended objectives.

3.1.2 Terms

- **PDF Retrieval** – Extraction of text and metadata from PDF files.
- **RAG** – Retrieval-Augmented Generation, combining search with generative AI.
- **ChromaDB** – Vector database for embedding-based document retrieval.
- **LangChain** – Framework for integrating LLMs with external data sources.
- **FastAPI** – Backend framework used to build RESTful APIs and WebSocket connections.

.

3.1.3 Intended Audience and Readers

The intended readers of this document include:

- **Development Team** – Responsible for implementing backend, database, and UI modules.
- **Project Guide and Evaluators** – Reviewing design, scope, and academic compliance.
- **Quality Assurance Team** – Testing functionality and reliability of the system.
- **End Users (Students & Learners)** – Benefiting from PDF-based question answering.

3.1.4 Product Scope

Askify is designed to bridge the gap between static PDF study material and interactive learning. The system enables:

- Secure PDF upload and storage.
- Text extraction from uploaded PDFs.
- Embedding and storage of extracted text in ChromaDB.
- Real-time Q&A through natural language queries.
- Personalized retrieval using RAG to ensure accurate responses.
- Scalable backend using FastAPI and WebSockets for multi-user access.
- Future enhancements may include integration with cloud storage, support for multimedia documents, and advanced exam-prep features like MCQ generation.

3.1.5 References

- LangChain documentation for NLP integration.
- ChromaDB documentation for embedding and retrieval.
- FastAPI official documentation.
- IEEE SRS Standard (IEEE 830-1998).
- Research papers on Retrieval-Augmented Generation (RAG).

3.2 General

3.2.1 Products

Askify is a web-based application with a backend service for PDF handling and Q&A. The product acts as a knowledge assistant for students and professionals by allowing interactive engagement with documents..

3.2.2 Product Features

- Upload and process PDF documents.
- Extract text and store in a SQLite/ChromaDB database.
- Perform semantic search using vector embeddings.
- Real-time Q&A through WebSocket endpoints.
- Simple web interface for uploading and querying.
- Logging and error handling for reliability.

3.2.3 User groups and characteristics

- Students: Upload notes/books and ask context-based questions.
- Teachers/Researchers: Retrieve specific content quickly.
- Guest Users: Limited access to try out system features.

3.2.4 Operating environment

- Backend hosted on POSTMAN.
- Python 3.11 environment with dependencies (LangChain, FastAPI, PyPDF2).

3.2.5 Design and Functionality

- Modular architecture for scalability.
- Efficient document embedding and retrieval system.
- Support for multi-user sessions with WebSockets.
- Designed for low latency responses with caching enabled..

3.2.6 User Information

Askify provides :

- Step-by-step guide for uploading PDFs.
- Documentation on how to ask effective questions.
- Help section for troubleshooting errors.

3.2.7 Assumptions and Dependencies

- Users have internet connectivity.
- PDFs are text-based (not purely scanned images unless OCR is added).
- System depends on the availability of LangChain API and ChromaDB.
- Hosting costs may vary with scale and number of active users..

3.3 External Interface Requirements

3.3.1 User Interface

- Through POSTMAN, user can GET/POST and ask questions in JSON form and Askify will give answers accordingly.

3.3.2 Hardware Interface

- Runs on cloud servers with scalable CPU/RAM.
- Local development tested on standard laptops/PCs.

3.3.3 Software Interface

- PyPDF2 for PDF extraction.
- LangChain + OpenAI/Gemini API for Q&A.
- ChromaDB for embeddings.
- FastAPI for API endpoints.

3.3.4 Communication

- WebSocket-based real-time communication for Q&A.
- REST endpoints for PDF upload and retrieval.
- System notifications via API logs.

3.4 Functions and requirements

3.4.1 Functional requirements

System Administrator

- Manage storage and monitor API usage.
- Handle authentication and error logs.

End User (Student/Teacher)

- Upload a PDF.
- Query content in natural language.
- Receive answers in real-time.
- View extracted text summary.

Database

- Store processed documents and embeddings.
- Maintain indexing for quick retrieval.

Communication Layer

- Handle WebSocket sessions.
- Ensure low-latency query responses.

Chapter 4

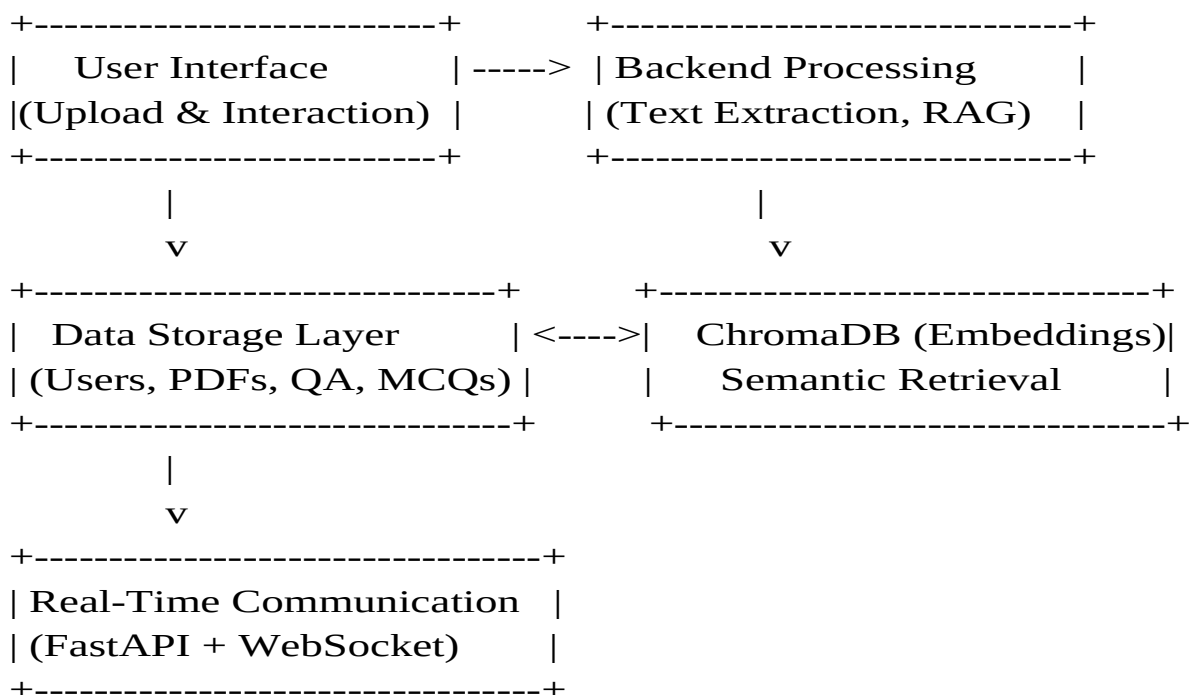
SYSTEM DESIGN

This chapter presents the design of the **Askify – PDF Question Answering System**. It covers the system architecture, providing an overview of the major components and their interactions, as well as the database structure used for storing documents, embeddings, and user data.

System design in Askify focuses on defining the architecture, modules, and interfaces required to support PDF upload, text extraction, semantic search, question answering, summarization, and MCQ generation. It translates user requirements into a detailed blueprint that guides the implementation phase, ensuring the system is efficient, scalable, and user-friendly.

4.1 System Architecture

The architecture of the Askify application can be illustrated using a high-level design as shown below:



Component Description:

- **User Interface:** This component acts as the primary interaction layer. Users interact with Askify by sending requests via Postman or similar API testing tools. Through these endpoints, users can upload PDFs, trigger summarization, ask questions, and retrieve generated answers or MCQs.
- **Document Processing Module:** Handles the uploaded PDFs by extracting text (using PyPDF2), chunking it into meaningful sections, and generating embeddings for semantic retrieval. It also powers summarization, question-answering, and MCQ generation through LangChain and RAG.

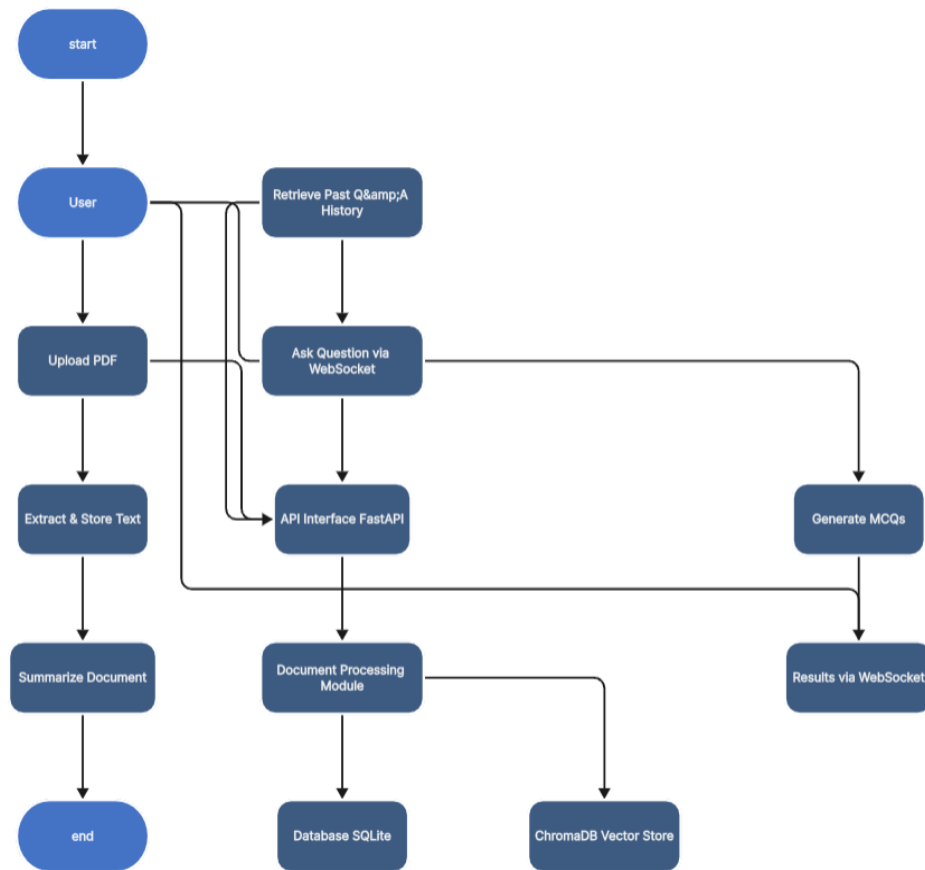


Figure 4.1: UML Diagram

- **Data Storage:** Stores user data, uploaded PDF metadata, extracted text, and embeddings. SQLite is used for structured storage, while ChromaDB manages high-dimensional vector embeddings for fast semantic search.
- **Real-Time Communication Layer (WebSocket):** Ensures continuous, low-latency communication between the backend and the client. This allows users to query PDFs and receive answers in real-time, simulating a conversational flow despite being API-based.

Data Stream:

- Users interact with the system via Postman or an API client, sending a request to upload a PDF document.
- The API interface (FastAPI endpoint) receives the uploaded file and forwards it to the document processing module.
- The Document Processing Module extracts text from the PDF, chunks it into sections, and generates vector embeddings.
- The extracted text and embeddings are securely stored in the SQLite database and ChromaDB for future retrieval.

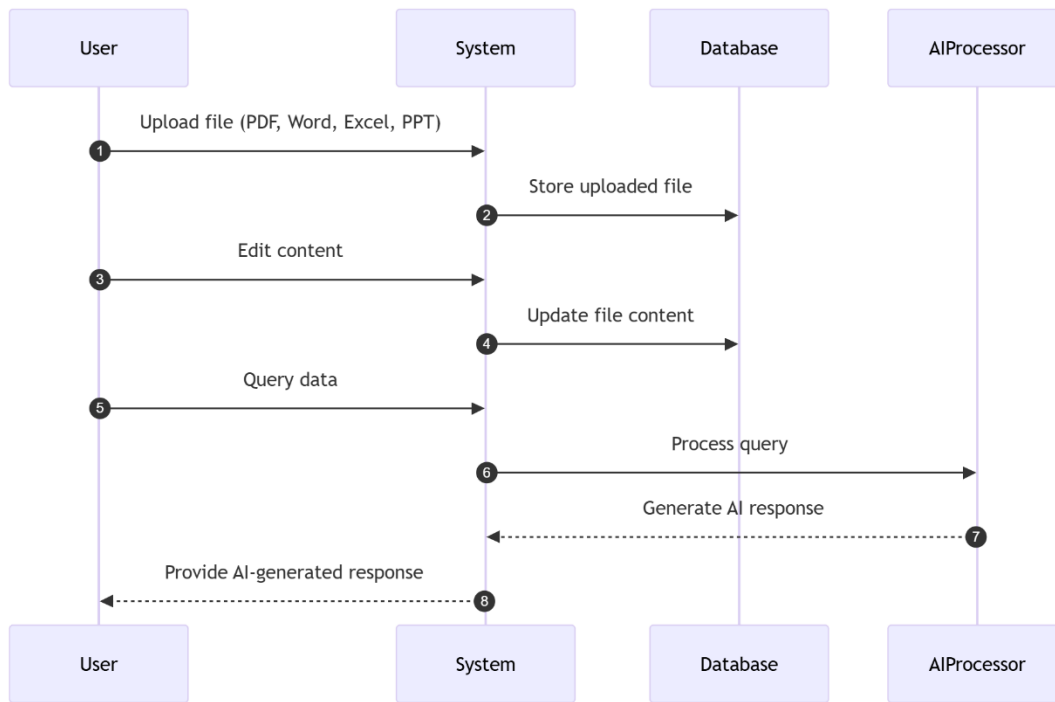


Figure 4.2: Sequence Diagram

- The client opens a WebSocket connection to `ws://.../ws/question_answer/<document_id>`.
- The client sends a natural-language question over the WebSocket.
- The backend validates the document ID and session; if valid, it fetches the document's chunk indices and metadata.
- The retriever queries ChromaDB with the question embedding and returns the top-k relevant chunks.
- The LangChain RAG pipeline constructs a prompt with the retrieved chunks and generates a grounded answer.
- The backend streams the answer (and optional citations/chunk refs) back to the client over the WebSocket in real time.
- The interaction (question, answer, timestamps) is logged in QA_History for later review/analytics.

4.2 User Interface Design

Askify does not provide a traditional web-based interface. Instead, it exposes a set of API endpoints that act as the primary user interface. Users interact with the system through tools such as Postman, cURL, or any HTTP/WebSocket client. These endpoints serve as the “resources” that enable document upload, processing, and retrieval of results.

- **PDF Upload Endpoint:** Allows users to send a PDF file to the system. On successful upload, the API responds with a unique document ID that will be used in subsequent operations.
- **Summarization Endpoint:** Provides concise and detailed summaries of the uploaded PDF. Users can specify the type of summary they need through request parameters.
- **Question Answering WebSocket:** A persistent WebSocket connection that lets users ask questions in real time. Questions are answered based on the content of the uploaded PDF, and responses are streamed back to the client.
 - **MCQ Generation Endpoint:** Generates multiple-choice questions (MCQs) from the document. The response includes questions, options, and correct answers
 - **History Retrieval Endpoint:** Enables users to fetch their previously uploaded documents, summaries, questions, and answers for review.

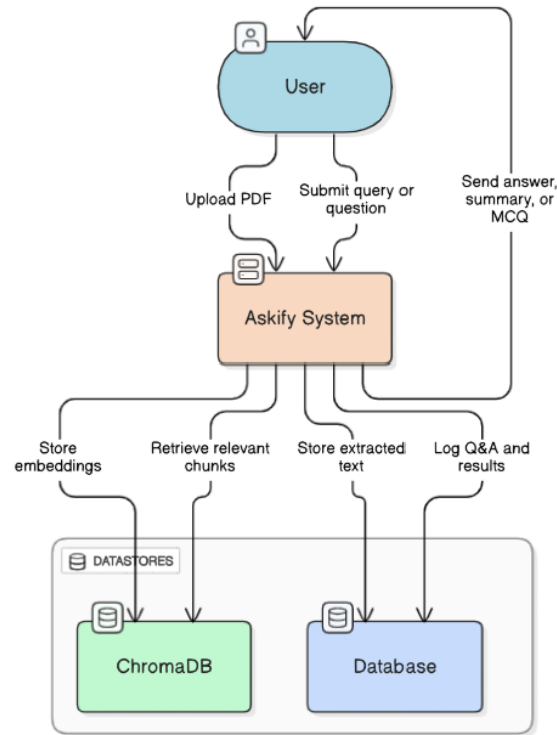


Figure 4.3: DFD Level-0

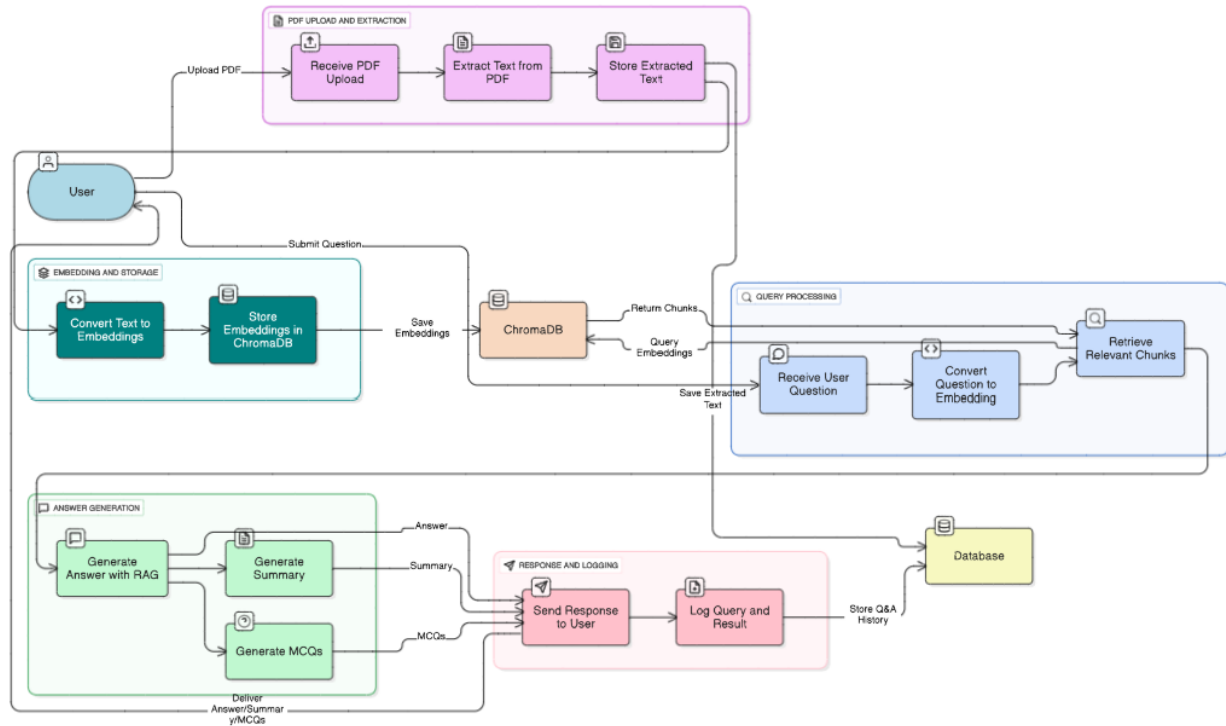


Figure 4.4: DFD Level-1

4.3 Database Design

Database architecture defines the storage structure, products, and history of value of the data used.

Below is an example of a relational database architecture:

Table 4.1: Database Schema

Table	Column	Data Type
Users	user_id (PK), username, email, password	INT PRIMARY KEY, VARCHAR(255) UNIQUE
Extracted_Text	text_id (PK), pdf_id (FK), content, embedding_id	INT PRIMARY KEY, INT FOREIGN KEY REFERENCES PDF_Documents(pdf_id), TEXT, VARCHAR(255)
Summaries	summary_id (PK), pdf_id (FK), short_summary, detailed_summary	INT PRIMARY KEY, INT FOREIGN KEY REFERENCES PDF_Documents(pdf_id), TEXT

Description:

- The 'Users' table stores details of registered users, including unique usernames and email addresses, With passwords always securely hashed before being stored. It is a good practice to security.
- The 'Extracted_Text' table contains text chunks obtained from uploaded PDFs, each linked to a specific document and associated with its embedding ID for semantic retrieval and efficient question answering.
- The 'Summaries' table stores concise and detailed summaries generated for each uploaded PDF, allowing users to quickly access important highlights without reading the entire file.

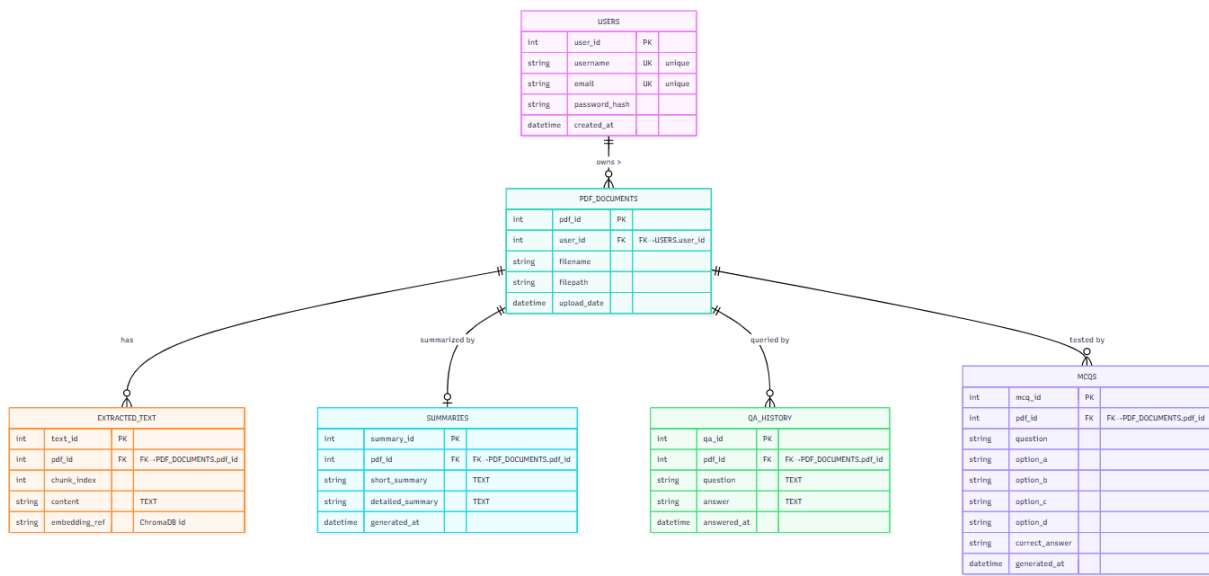


Figure 4.5: ER Model

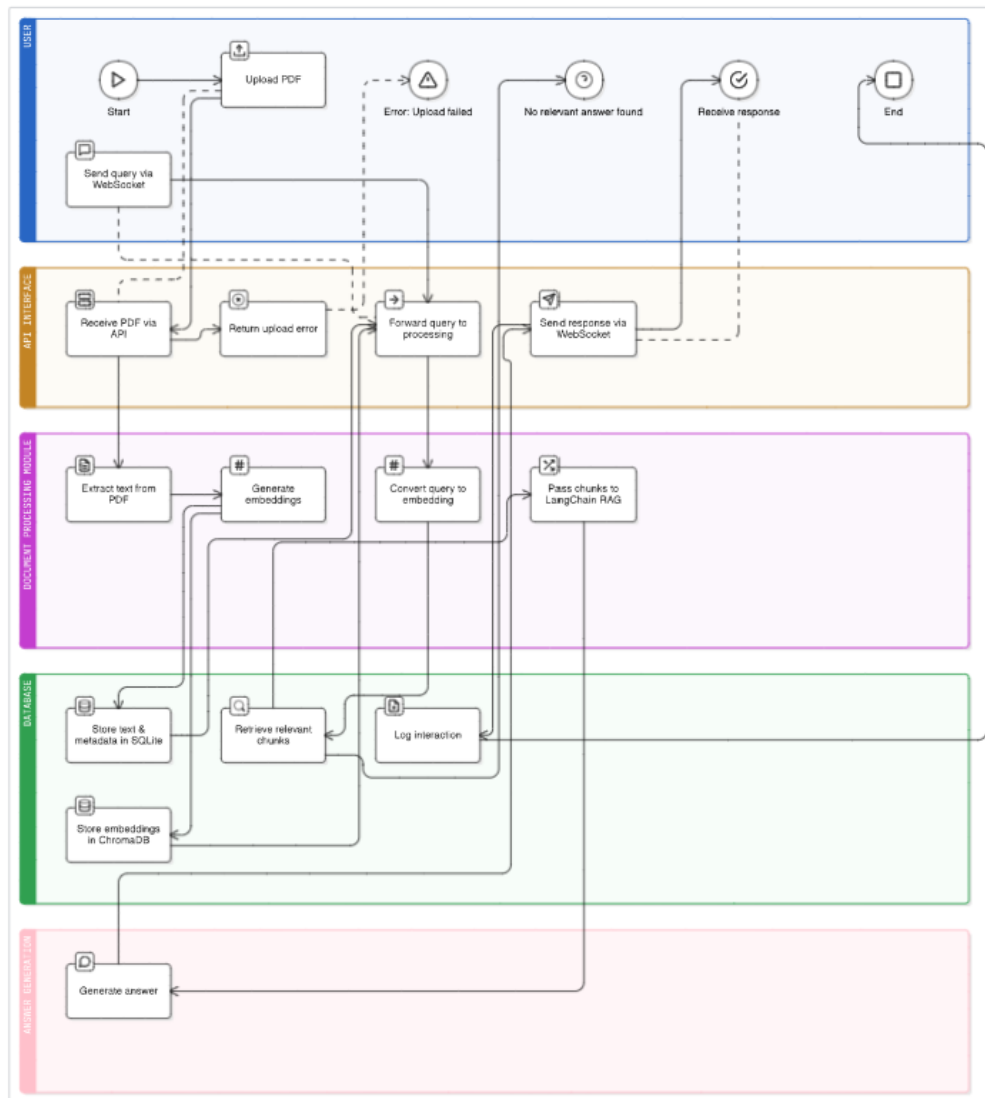


Figure 4.6: Activity Diagram

Chapter 5

METHODOLOGY

This chapter outlines the methodology adopted for the development of Askify – PDF Question Answering System. It describes the technology stack, the development workflow, and the testing methods applied to ensure the system meets both functional and non-functional requirements.

5.1 Technology Stack

The development of Askify incorporates the following technologies:

- **Programming Language:** Python – Selected for its simplicity, flexibility, and rich ecosystem of libraries supporting natural language processing, machine learning, and API development.
- **Web Framework:** FastAPI – A modern and high-performance Python web framework used to design the backend. It provides RESTful endpoints for PDF upload, summarization, and MCQ generation, as well as WebSocket-based communication for real-time question answering.
- **PDF Processing Library:** PyPDF2 – Used to parse and extract text from uploaded PDF documents, enabling downstream processing and content chunking.
- **Natural Language Processing:** LangChain – A framework that integrates large language models (LLMs) with external knowledge sources. LangChain powers Askify’s question-answering and summarization modules.

- **Vector Database:** ChromaDB – Responsible for storing and retrieving vector embeddings of document chunks. It supports semantic similarity search to ensure relevant context is provided to the language model during Q&A.
- **Database Management System (DBMS):** SQLite – Used as the relational database for managing structured data such as user information, uploaded PDF metadata, summaries, Q&A history, and MCQs.
- **Real-Time Communication:** WebSockets – Enables interactive Q&A sessions between users and the system, providing low-latency responses.
- **Environment Management:** .env and API Keys – Secure storage and configuration of credentials such as API keys for LLM services (e.g., Gemini or LangChain API).

5.2 Development Process

The development of Askify follows a structured, modular workflow to ensure scalability, maintainability, and user-focused outcomes. The process is divided into the following stages:

1. PDF Upload and Processing:

- Users interact with the system via API requests (e.g., Postman) to upload PDF documents.
- The backend, built on FastAPI, validates the input and stores the PDF metadata in the database.
- The PyPDF2 library is used to parse and extract text from the PDF, splitting it into manageable chunks for downstream tasks.

2. Embedding Generation and Storage:

- Extracted text chunks are converted into vector embeddings using pre-trained models integrated through LangChain.
- These embeddings are stored in ChromaDB, enabling efficient semantic search and retrieval.
- Metadata and text references are stored in SQLite for structured management.

3. Question Answering (Q&A):

- Users send natural language queries through the WebSocket endpoint.
- The backend generates embeddings for the query and retrieves the most relevant chunks from ChromaDB.
- The LangChain RAG pipeline combines the retrieved context with a large language model to generate an accurate, context-aware answer.
- The answer is returned in real time through the WebSocket connection and simultaneously logged in the QA_History table.

4. Summarization:

- The system generates both concise and detailed summaries of the uploaded PDF using LangChain's summarization chains.
- These summaries are stored in the Summaries table and can be retrieved by the user on request.

5.3 Testing and Evaluation

Thorough testing was conducted during the development lifecycle of Askify to ensure the system's accuracy, reliability, and robustness. The following testing methods were applied:

- **PDF Parsing Accuracy:** Unit tests were performed to confirm that the **PyPDF2 module** correctly extracted text from different types of PDF documents. Edge cases, such as scanned or corrupted PDFs, were also tested to validate error handling.
- **Embedding and Retrieval Validation:** Tests were carried out to ensure that **text chunks and embeddings** stored in **ChromaDB** matched the original PDF content. Retrieval accuracy was evaluated by comparing returned chunks against user queries to verify semantic similarity.
- **Q&A Functionality Testing:** The **WebSocket endpoint** was tested for reliability and low latency. Queries of varying complexity were submitted, and responses were evaluated for correctness, relevance, and consistency with the PDF content.
- **Summarization and MCQ Evaluation:** Summaries were tested for coherence and coverage of important topics. Generated MCQs were validated by human evaluators to ensure question clarity, relevance, and correctness of answer options.
- **Database Integrity:** Verification systems confirmed that **SQLite tables** (Users, Documents, QA_History, Summaries, MCQs) handled data insertion, updates, and retrieval correctly. Referential integrity between tables was maintained, and automated tests ensured no data loss during operations.
- **Performance Testing:** The system was tested under simulated concurrent requests to ensure scalability and responsiveness. The backend consistently maintained acceptable response times for PDF processing and query answering.

The combination of these testing methods ensured that Askify operates reliably, delivers accurate answers, and provides a smooth API-based interaction for end users.

Chapter 6

IMPLEMENTATION

6.1 Introduction

This section describes the implementation process of Askify – PDF Question Answering System. It explains the stages involved in building the system, including the setup of APIs, integration of natural language processing frameworks, and database management. The implementation ensures that the system is capable of handling PDF uploads, extracting text, generating embeddings, and delivering real-time answers, summaries, and MCQs.

6.2 Technology Stack

The Askify system was implemented using the following technologies:

- **Programming Language:** Python – core development language.
- **Web Framework:** FastAPI – for building REST APIs and WebSocket endpoints.
- **PDF Processing:** PyPDF2 – for parsing and extracting text from PDF documents.
- **NLP Framework:** LangChain – for summarization, question answering, and MCQ generation using Retrieval-Augmented Generation (RAG).
- **Database:** SQLite – for structured data storage (users, PDFs, summaries, Q&A history, MCQs).
- **Vector Database:** ChromaDB – for managing embeddings and semantic similarity search.
- **Real-Time Communication:** WebSockets – for handling interactive Q&A between users and the backend.
- **Environment Management:** .env for securely storing API keys and configurations.

6.3 API-Based Interface

Unlike traditional web applications, Askify does not rely on a dedicated front end. Instead, the system provides API endpoints that serve as the primary interface for users. Tools such as Postman or cURL are used to interact with the system.

Key API resources include:

- **/upload Endpoint:** Allows PDF upload and returns a document ID for reference.
- **/summarize/{doc_id} Endpoint:** Generates and retrieves both short and detailed summaries.
- **/generate_mcqs/{doc_id} Endpoint:** Creates multiple-choice questions from document.
- **/ws/question_answer/{doc_id} WebSocket:** Provides real-time question answering by retrieving relevant text chunks and generating context-aware answers.
- This approach ensures simplicity, flexibility, and scalability, while maintaining a developer-friendly workflow.

6.4 Back End Development

The backend is the core of Askify, designed to handle document processing and natural language queries. The following modules were implemented:

- **PDF Processing Module:** Extracts text from PDFs and prepares it for downstream tasks.
- **Embedding Module:** Converts text chunks into embeddings using LangChain-supported models and stores them in ChromaDB.
- **Summarization Module:** Automatically generates concise and detailed summaries using transformer-based summarization chains.
- **Question Answering Module:** Retrieves relevant text chunks from ChromaDB and applies RAG to generate answers.
- **MCQ Generation Module:** Creates multiple-choice questions with options and correct answers, stored in the MCQs table.
- **Database Integration:** SQLite manages user accounts, documents, summaries, MCQs, and Q&A logs. Relationships between these tables ensure data consistency.
- **WebSocket Handler:** Ensures interactive communication between the backend and the client, streaming answers in real time.

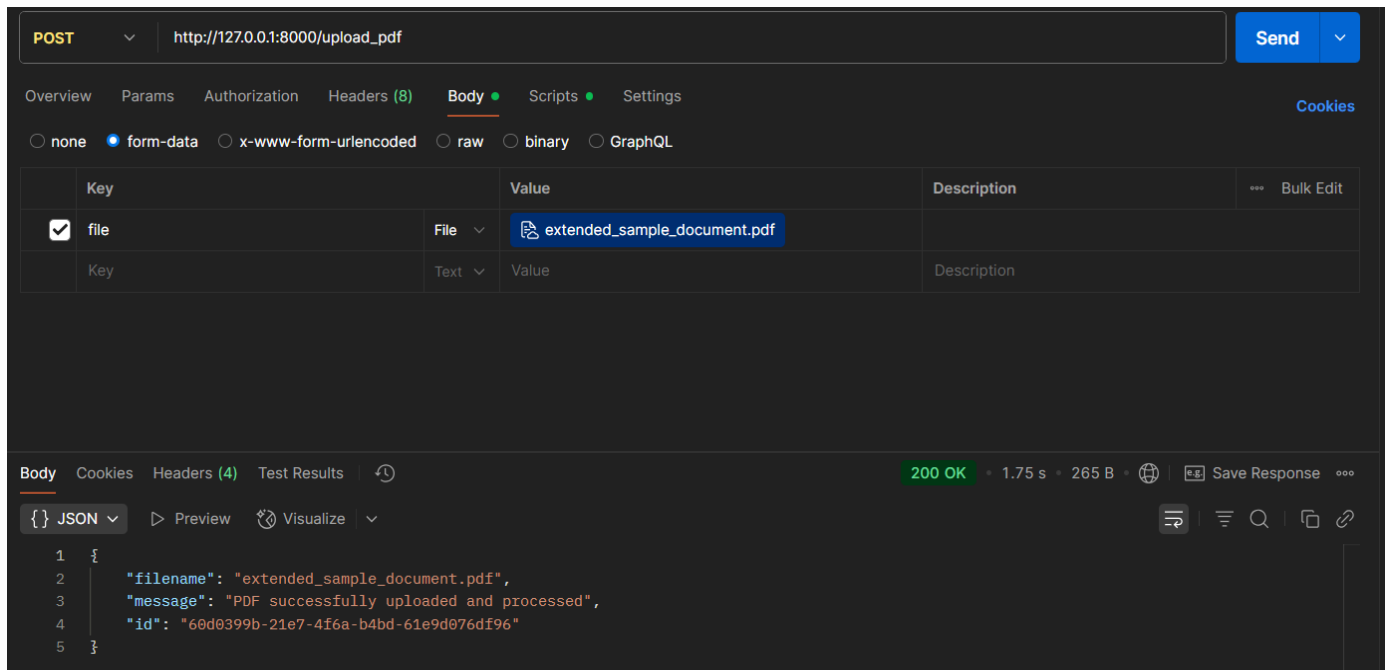


Figure 6.1: PDF Upload via Postman

Description: Screenshot of the Postman interface showing the POST `/upload` endpoint. The request includes a sample PDF file attached, and the response displays a success message along with a unique document ID. This highlights the first step of system interaction.

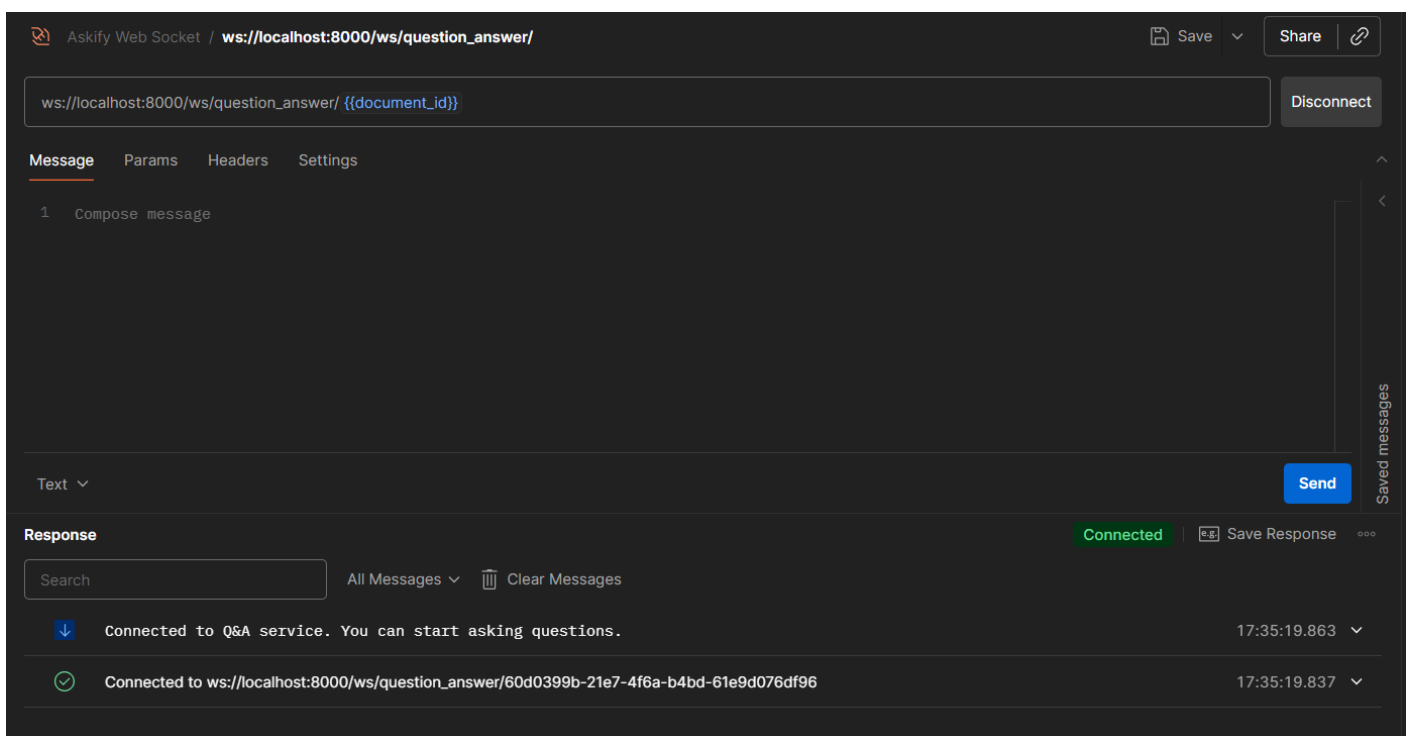


Figure 6.2: Connect Localhost to Askify

Description: To connect Askify, `ws://localhost:8000/ws/question_answer/{{document_id}}` enter this path in the url section and hit the button Connect. Meanwhile, we got a success message like Connected to `ws://localhost:8000/ws/question_answer/60d0399b-21e7-4f6a-b4bd-61e9d076df96`

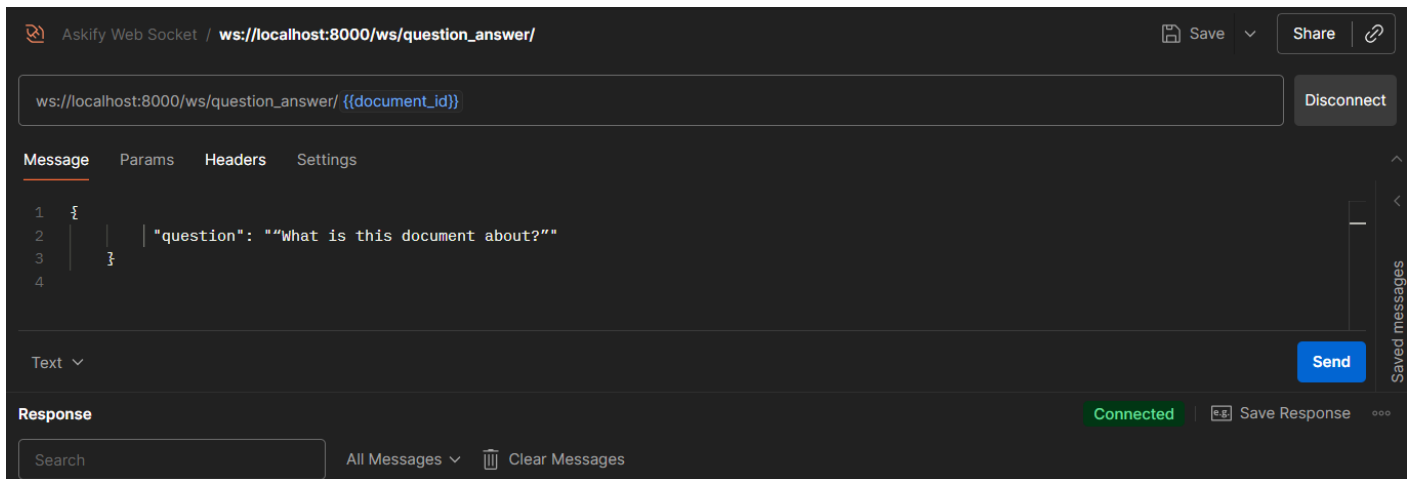


Figure 6.3: Real-Time Question Answering (WebSocket)

Description: Demonstration of the WebSocket connection in action. The figure shows the client sending a natural language question (e.g., “What is this document about?”) and receiving a context-aware answer streamed back from the backend. This emphasizes Askify’s interactive Q&A capability..

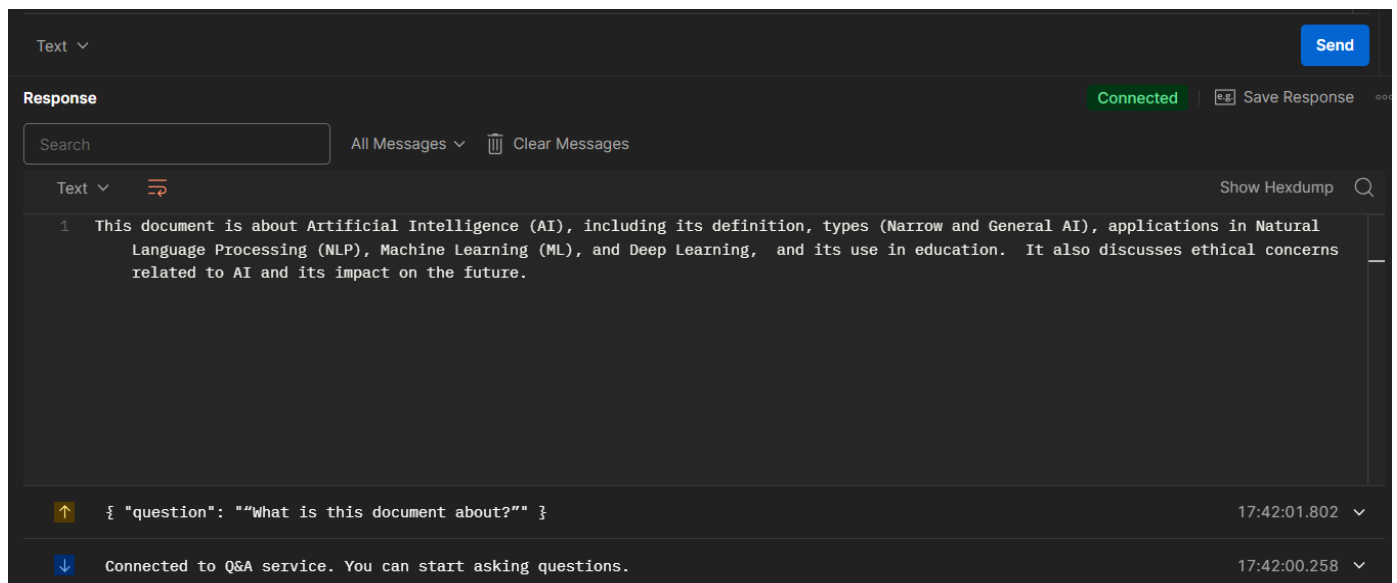


Figure 6.4: Summarization Response via API

Description: Screenshot of Postman showing a request to the `/summarize/{doc_id}` endpoint. The response displays both a short and a detailed summary generated by the system for the uploaded PDF. This demonstrates Askify’s summarization functionality.

6.5 Document Processing and RAG Implementation

The core functionality of Askify lies in its ability to process uploaded PDF documents and support real-time question answering. The implementation involved:

- Extracting text from PDFs using PyPDF2, ensuring support for multi-page and structured documents.
- Splitting the extracted text into smaller, semantically meaningful chunks for efficient retrieval.
- Generating embeddings for these chunks using LangChain and storing them in ChromaDB.
- Enabling Retrieval-Augmented Generation (RAG) pipelines, where user queries are embedded, matched against stored vectors, and combined with relevant chunks to produce context-aware answers.

This pipeline ensures that every query is answered with precise information derived directly from the uploaded documents.

6.6 Data Processing and Insights

The data extracted from PDFs undergoes multiple processing stages to produce meaningful outputs for the user. These include:

- **Summarization:** Generating both short and detailed summaries of documents to provide users with key takeaways.
- **Question Answering:** Returning direct, context-driven answers to natural language queries using the stored document embeddings.
- **MCQ Generation:** Producing multiple-choice questions with answer keys to support exam preparation and active learning.
- **History Logging:** Maintaining records of all queries, summaries, and MCQs for users to revisit and analyze.

6.7 Challenges and Solutions

- **Handling Diverse PDF Structures:** PDFs vary in formatting, such as scanned images, tables, or inconsistent layouts. To overcome this, preprocessing and text-cleaning strategies were applied to improve extraction quality.
- **Embedding Storage and Retrieval Efficiency:** Large documents generate high volumes of text chunks. Efficient indexing in ChromaDB was implemented to speed up retrieval without sacrificing accuracy.
- **Real-Time Response via WebSocket:** Ensuring low latency during Q&A was a challenge due to model inference time. This was mitigated by optimizing the RAG pipeline and streaming partial responses when applicable.
- **Scalability Concerns:** Since more users and larger PDFs increase database load, caching strategies and database optimization techniques were planned to maintain consistent performance.
- **Ethical and Responsible AI Use:** Ensuring that answers remain faithful to the PDF content was critical. Guardrails in LangChain were configured to minimize hallucinations and prevent misleading outputs.

Chapter 7

TESTING

7.1 Introduction

Testing is a critical step in verifying the reliability and accuracy of Askify. The objective is to ensure that the system correctly extracts text from uploaded PDFs, generates embeddings, provides accurate answers in real time, and stores summaries and MCQs reliably. Testing also validates that the API endpoints and WebSocket services function consistently across different use cases..

7.2 Unit Testing

Unit tests were performed on individual modules of the system to confirm their correctness and robustness. Specific focus areas included:

- **PDF Extraction Functions:** Validating that text is consistently and accurately extracted from PDFs of varying formats (multi-page, scanned, or structured).
- **API Endpoints:** Ensuring that REST endpoints such as `/upload`, `/summarize/{doc_id}`, and `/generate_mcqs/{doc_id}` accept valid inputs and return correct structured outputs.
- **WebSocket Q&A Module:** Verifying that questions sent via the WebSocket are correctly processed and answers are returned without errors.
- **Database Operations:** Testing CRUD (Create, Read, Update, Delete) operations on SQLite tables such as Users, PDF_Documents, Summaries, and QA_History.

7.3 Integration Testing

Integration testing was conducted to ensure smooth interaction between the core components of Askify. This involved:

- **PDF Upload and Database Storage:** Confirming that PDFs uploaded via the API are parsed correctly, and the extracted text and metadata are stored in SQLite without loss.
- **Embeddings and ChromaDB Retrieval:** Testing the flow from extracted text → embedding generation → storage in ChromaDB, and validating that queries retrieve the most relevant chunks.
- **LangChain RAG with ChromaDB:** Verifying that retrieved document chunks are correctly combined with the LLM to generate accurate, context-based answers.
- **Summarization and MCQ Modules:** Ensuring that outputs (summaries and MCQs) are linked to the correct document ID and retrievable through the API.
- **WebSocket End-to-End Flow:** Testing the user interaction of uploading a PDF, sending a question via WebSocket, and receiving a context-aware response in real time.

```

HP@Shannu-s-HP-15-S MINGW64 ~/OneDrive/Desktop/Askify-main/Askify-main (main)
$ pip install -r requirements.txt
Requirement already satisfied: fastapi in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 1)) (0.115.8)
Requirement already satisfied: uvicorn in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 2)) (0.34.0)
Requirement already satisfied: sqlalchemy in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 3)) (2.0.38)
Requirement already satisfied: python-multipart in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 4)) (0.0.20)
Requirement already satisfied: pydantic in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 5)) (2.10.6)
Requirement already satisfied: PyMuPDF in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 6)) (1.25.3)
Requirement already satisfied: pypdf in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 7)) (5.2.0)
Requirement already satisfied: pypdf2 in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 8)) (3.0.1)
Requirement already satisfied: langchain in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 9)) (0.3.18)
Requirement already satisfied: langchain-community in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 10)) (0.3.17)
Requirement already satisfied: langchain-google-genai in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 11)) (2.0.9)
Requirement already satisfied: python-dotenv in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 12)) (1.0.1)
Requirement already satisfied: pytest in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 13)) (8.3.4)
Requirement already satisfied: httpx in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 14)) (0.28.1)
Requirement already satisfied: pytest-asyncio in c:\users\hp\onedrive\desktop\askify-main\askify-main\venv\lib\site-packages (from -r requirements.txt (line 15)) (0.25.3)

```

Figure 7.1: Dependency Installation (pip install -r requirements.txt)

Description: This figure shows the terminal output of installing all required dependencies for Askify using the command `pip install -r requirements.txt`. It verifies that the environment setup is correct and all necessary libraries (FastAPI, LangChain, ChromaDB, PyPDF2, etc.) are successfully installed before running the system.

```

HP@Shannu-s-HP-15-S MINGW64 ~/OneDrive/Desktop/Askify-main/Askify-main (main)
$ uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\HP\\OneDrive\\Desktop\\Askify-main\\Askify-main']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [32104] using WatchFiles
C:\\Users\\HP\\OneDrive\\Desktop\\Askify-main\\Askify-main\\rag.py:5: LangChainDeprecationWarning: Importing FAISS from langchain.vectorstores is deprecated. Please replace deprecated imports:

>> from langchain.vectorstores import FAISS

with new imports of:

>> from langchain_community.vectorstores import FAISS
You can use the langchain cli to **automatically** upgrade many imports. Please see documentation here <https://python.langchain.com/docs/versions/v0_2/>
    from langchain.vectorstores import FAISS
INFO: Started server process [22912]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:43165 - "POST /upload_pdf HTTP/1.1" 200 OK
Error loading existing vector store: The de-serialization relies loading a pickle file. Pickle files can be modified to deliver a malicious payload that results in execution of arbitrary code on your machine. You will need to set `allow_dangerous_deserialization` to `True` to enable deserialization. If you do this, make sure that you trust the source of the data. For example, if you are loading a file that you created, and know that no one else has modified the file, then this is safe to do. Do not set this to `True` if you are loading a file from an untrusted source (e.g., some random site on the internet.).
INFO:faiss.loader:Loading faiss with AVX2 support.
INFO:faiss.loader:Successfully loaded faiss with AVX2 support.
INFO:faiss:Failed to load GPU Faiss: name 'GpuIndexIVFFlat' is not defined. Will not load constructor refs for GPU indexes.
Vector store saved locally.
INFO:routers.question_answer:Chat service initialized for document: 60d0399b-21e7-4f6a-b4bd-61e9d076df96
INFO: ('127.0.0.1', 52732) - "WebSocket /ws/question_answer/60d0399b-21e7-4f6a-b4bd-61e9d076df96" [accepted]
INFO:routers.question_answer:Client 60d0399b-21e7-4f6a-b4bd-61e9d076df96_1756017317.436781 connected. Active connections: 1
INFO: connection open

```

Figure 7.2: Running the Server with Uvicorn

Description: This figure illustrates the execution of the command `uvicorn main:app --reload`, which starts the Askify backend server. The terminal output confirms that the FastAPI application is running successfully and is accessible at `http://127.0.0.1:8000`.

.\

7.4 User Acceptance Testing

User Acceptance Testing (UAT) was conducted by engaging a group of students and testers who interacted with Askify through API requests and WebSocket connections. The objective was to validate whether the system met real-world requirements such as accurate PDF text extraction, reliable summarization, context-aware Q&A, and meaningful MCQ generation.

7.5 Test Cases and Results

Below are the key test cases that were executed:

- **Test Case 1: PDF Upload and Extraction**
 - **Objective:** Verify that PDFs are uploaded and text is extracted correctly.
 - **Steps:** Upload a sample PDF using the /upload API.
 - **Expected Result:** Extracted text chunks should be stored in SQLite and embeddings generated in ChromaDB .
 - **Actual Result:** Passed.

- **Test Case 2: Real-Time Question Answering**
 - **Objective:** Ensure accurate answers are returned via WebSocket.
 - **Steps:** Connect to /ws/question_answer/{doc_id} and ask questions about the uploaded document.
 - **Expected Result:** System should return context-aware answers from the relevant PDF content.
 - **Actual Result:** Passed.

- **Test Case 3: Database Performance**
 - **Objective:** Assess MongoDB's performance with large datasets.
 - **Steps:** Insert and query substantial volumes of scraped data.
 - **Expected Result:** Quick response times without noticeable latency.
 - **Actual Result:** Passed.

7.6 Performance Testing

Performance testing for Askify was carried out across key areas to evaluate efficiency and scalability:

- **PDF Processing Speed:** The system was tested with PDFs of varying sizes (small documents <1 MB, medium ~10 MB, and large academic papers >50 MB). The extraction process consistently completed within acceptable limits, with larger files taking slightly longer but still manageable.
- **Embedding and Retrieval Latency:** ChromaDB performance was assessed under high query loads. Embedding generation and vector searches were optimized to ensure quick retrieval of relevant chunks.
- **System Response Time:** Real-time queries sent via the WebSocket endpoint were tested with multiple concurrent users. The average latency for Q&A responses remained within 2–3 seconds for medium-sized documents, ensuring interactive usability.

7.7 Bug Tracking and Resolution

Bugs encountered during development and testing were tracked using an internal logging system. The most common issues included:

- **PDF Extraction Failures:** Some PDFs with complex formatting or scanned pages failed to extract properly. These were mitigated by refining preprocessing logic and adding error-handling routines.
- **Incorrect Embedding Matches:** In early testing, some irrelevant text chunks were retrieved. The issue was resolved by tuning the embedding similarity thresholds.
- **WebSocket Disconnections:** Occasional connection drops during Q&A were fixed by implementing retry mechanisms and keeping the connection alive with heartbeat signals.

7.8 Final Testing and Validation

A final validation phase was performed before deployment to ensure that all features worked as intended under realistic usage conditions. This included:

- Running the system with large PDFs and multiple users to validate stability under load.
- Verifying that summaries accurately captured key document information.
- Confirming that Q&A answers remained contextually relevant across diverse queries.

The results indicated that Askify successfully met the functional and non-functional requirements.

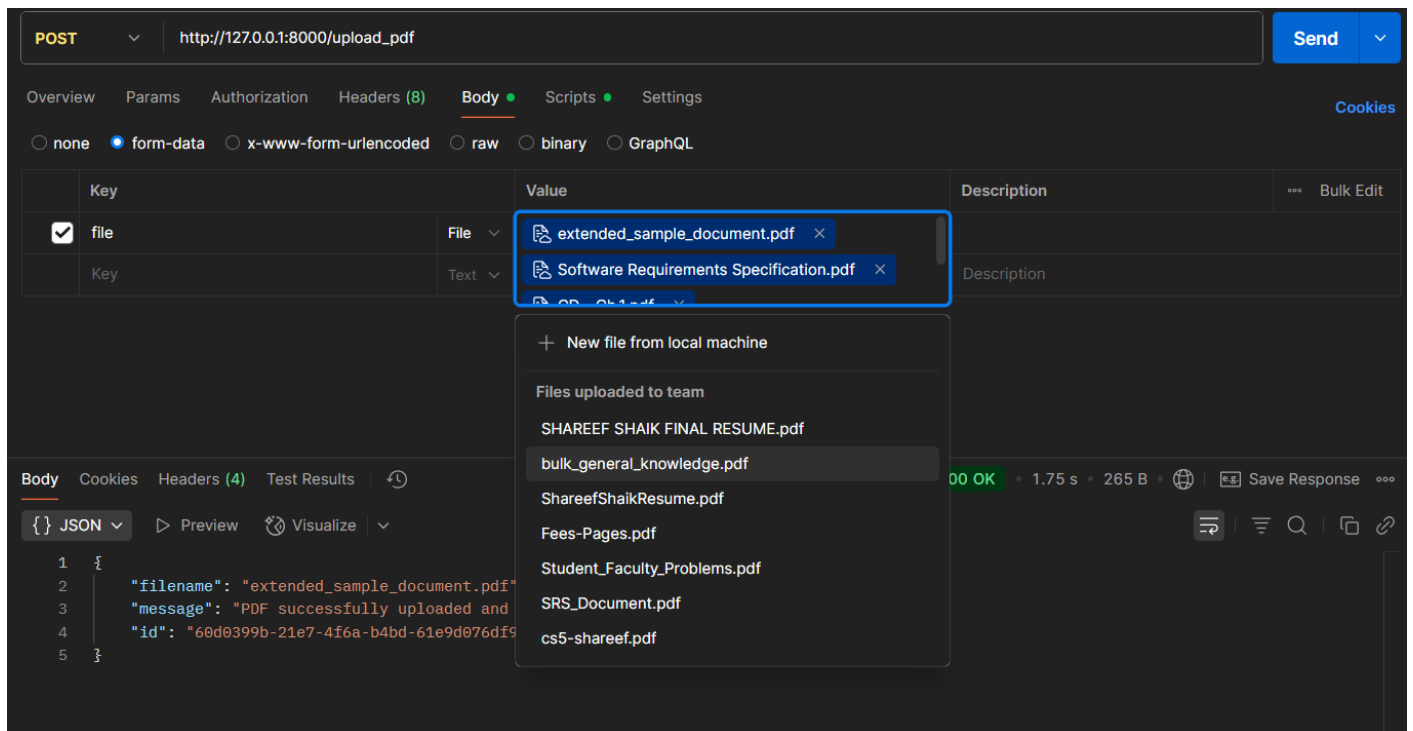


Figure 7.4: Final Performance and Stress Testing

Description: This figure shows the final round of testing where Askify was stress-tested with multiple large PDF uploads and concurrent user queries. The test validated that the system could handle real-time question answering and summarization under peak load conditions, maintaining acceptable response times and stable WebSocket connections.

Chapter 8

CONCLUSION

8.1 Summary of Results

In this project, we successfully developed a backend service named Askify, designed to extract text from PDF documents and enable intelligent question answering, summarization, and MCQ generation. By leveraging advanced NLP frameworks such as LangChain and embedding storage with ChromaDB, the system allows users to interact with their documents in real time. The integration of FastAPI and WebSockets provides a seamless way to upload documents, ask questions, receive summaries, and generate exam-oriented practice questions.

8.2 Key Findings

During the development and testing process, several key findings emerged:

- Retrieval-Augmented Generation (RAG) proved highly effective in delivering context-aware answers by combining embeddings with language model outputs.
- Proper text preprocessing and chunking of PDFs were critical to ensure accurate retrieval and reduce irrelevant results.
- The use of ChromaDB for similarity search enhanced system efficiency by significantly reducing query latency.
- Summarization and MCQ generation were identified as highly valuable features for students preparing for exams, providing both quick revisions and practice material.

8.3 Challenges

The project encountered various challenges that required adaptive solutions:

- Handling diverse PDF formats, especially scanned or poorly structured documents, often led to incomplete text extraction.
- Maintaining response accuracy and avoiding hallucinations in the Q&A module required fine-tuning embedding similarity thresholds.
- Optimizing performance to ensure low-latency responses during real-time WebSocket sessions was a constant concern.
- Designing MCQs that were both relevant and balanced in difficulty required iterative improvements in generation logic.

8.4 Lessons Learned

This project offered valuable insights into the practical implementation of NLP-driven document systems. Key lessons included:

- The importance of robust error handling during PDF extraction and query processing to ensure system reliability.
- The significance of combining symbolic methods (databases, embeddings) with generative AI for accurate results.
- A deeper understanding of ethical AI usage, particularly regarding the need for outputs to remain faithful to the original document to avoid misinformation.

8.5 Future Directions

Several opportunities exist for further research and enhancement of Askify:

- Integration of Optical Character Recognition (OCR) to handle scanned or image-based PDFs.
- Improving summarization techniques by incorporating abstractive summarization models for more human-like outputs.
- Expanding beyond PDFs to support additional file formats such as Word documents and PowerPoint presentations.
- Implementing user-facing dashboards or mobile applications for broader accessibility.
- Scaling the system to support concurrent usage by large numbers of students and institutions.

8.6 Conclusion

In summary, the successful development of Askify demonstrates the potential of AI-driven systems to transform document interaction. By enabling PDF-based question answering, summarization, and exam-focused MCQ generation, the system provides students and professionals with a powerful tool for learning and revision. The project highlights the effectiveness of combining FastAPI, LangChain, and ChromaDB into a robust backend service, setting the foundation for future improvements and broader applications.

Chapter 9

FUTURE WORK

While the Askify project has concluded successfully, there are numerous opportunities for future enhancements and expansions that could further increase its impact and usability. These include:

- **Research Publication:** Building on the innovative approach and positive results of Askify, there is strong potential to publish a research paper. Such work could explore the technical aspects of Retrieval-Augmented Generation (RAG), embedding-based document retrieval, and the educational applications of automated summarization and MCQ generation. It could also discuss ethical concerns related to AI-driven content generation and responsible system deployment.
- **Enhanced Language Models:** Future versions of Askify could integrate more advanced large language models (LLMs) to improve the accuracy and depth of summarization, question answering, and MCQ generation. Fine-tuning models for academic contexts could further enhance relevance and reliability.
- **Support for Multiple File Formats:** While the current system primarily supports PDFs, expanding to include Word documents, PowerPoint presentations, and scanned files with OCR would broaden its applicability and make it more versatile for students and professionals.
- **Mobile Application Development:** A mobile version of Askify could make the service more accessible, enabling users to upload documents, receive summaries, and interact with Q&A features on the go. Push notifications for updates (like generated summaries or new practice questions) could further improve engagement.
- **User Personalization:** Adding features such as personalized dashboards, saved document history, customizable summary lengths, and difficulty levels for MCQs could enhance user experience. Personalized study recommendations could also be introduced based on a user's activity and performance.
- **Improved Data Visualization:** Future versions could integrate visual dashboards for displaying key insights, such as summary highlights, question-answer distributions, and topic coverage maps. Visual analytics would help users better understand and interact with large documents.
- **Ethical and Responsible AI Practices:** As AI systems become more widely adopted, ensuring fairness, transparency, and faithfulness of generated content remains crucial. Future work will focus on implementing safeguards to minimize hallucinations, protect data privacy, and align the system with academic integrity principles.

Bibliography

- [1] Bernard, N., & Balog, K. (2025). A Systematic Review of Fairness, Accountability, Transparency, and Ethics in Information Retrieval. **ACM Computing Surveys**, 57(6), 1–29. Highlights ethical design principles in IR systems.
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. **NeurIPS**. Introduces RAG models that combine parametric and non-parametric memory for improved QA.
- [3] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. **arXiv preprint**. Integrates a latent retriever into language-model pre-training for enhanced QA accuracy.
- [4] Wu, Y., Zhao, Y., Hu, B., Minervini, P., Stenetorp, P., & Riedel, S. (2022). An Efficient Memory-Augmented Transformer for Knowledge-Intensive NLP Tasks. **arXiv preprint**. Proposes an EMAT model combining fast memory retrieval with expressiveness.
- [5] “Retrieval-augmented generation” (2025). **Wikipedia**. Explains RAG fundamentals and how it mitigates hallucination by grounding responses in external sources.
- [6] **Hoang, T. T. U., & Nguyen, V. A. (2025)**. PDF Retrieval Augmented Question Answering. Saarland University. Demonstrates how RAG enhances QA over multi-modal PDF data (text, images, tables) for complex queries.
- [7] **Wang, Z. (2025)**. Document Segmentation Matters for Retrieval-Augmented Generation. Findings of ACL. Highlights the critical role of chunking documents into meaningful units for effective RAG pipelines.

- [8] **Muludi, K. M., Fitria, K. M., Triloka, J., Sutedi (2024).** Retrieval-Augmented Generation Approach: Document Question Answering using Large Language Model. IJACSA, 15(3), 776–. Compares RAG against traditional QA systems using GPT-3.5-turbo, demonstrating higher precision and F1 scores in document-based QA.
- [9] **Allu, U., Ahmed, B., Tripathi, V. (2024).** Beyond Extraction: Contextualising Tabular Data for Efficient Summarisation by Language Models. arXiv preprint. Proposes enhanced handling of tables within PDFs for improved summarization in RAG systems.
- [10] **Siriwardhana, S., Weerasekera, R., Wen, E., Kaluarachchi, T., Rana, R., Nanayakkara, S. (2022).** Improving the Domain Adaptation of RAG Models for Open Domain QA. arXiv preprint. Introduces RAG-end2end that supports domain-specific adaptation through joint retriever/generator training.

